

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: Implement best security practices for IP, email, and web service using PGP,

S/MIME for enhancing email Security.CO (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A
Coding Challenge

Code of Conduct:

I **P.paneendra (192321072)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Paneendra

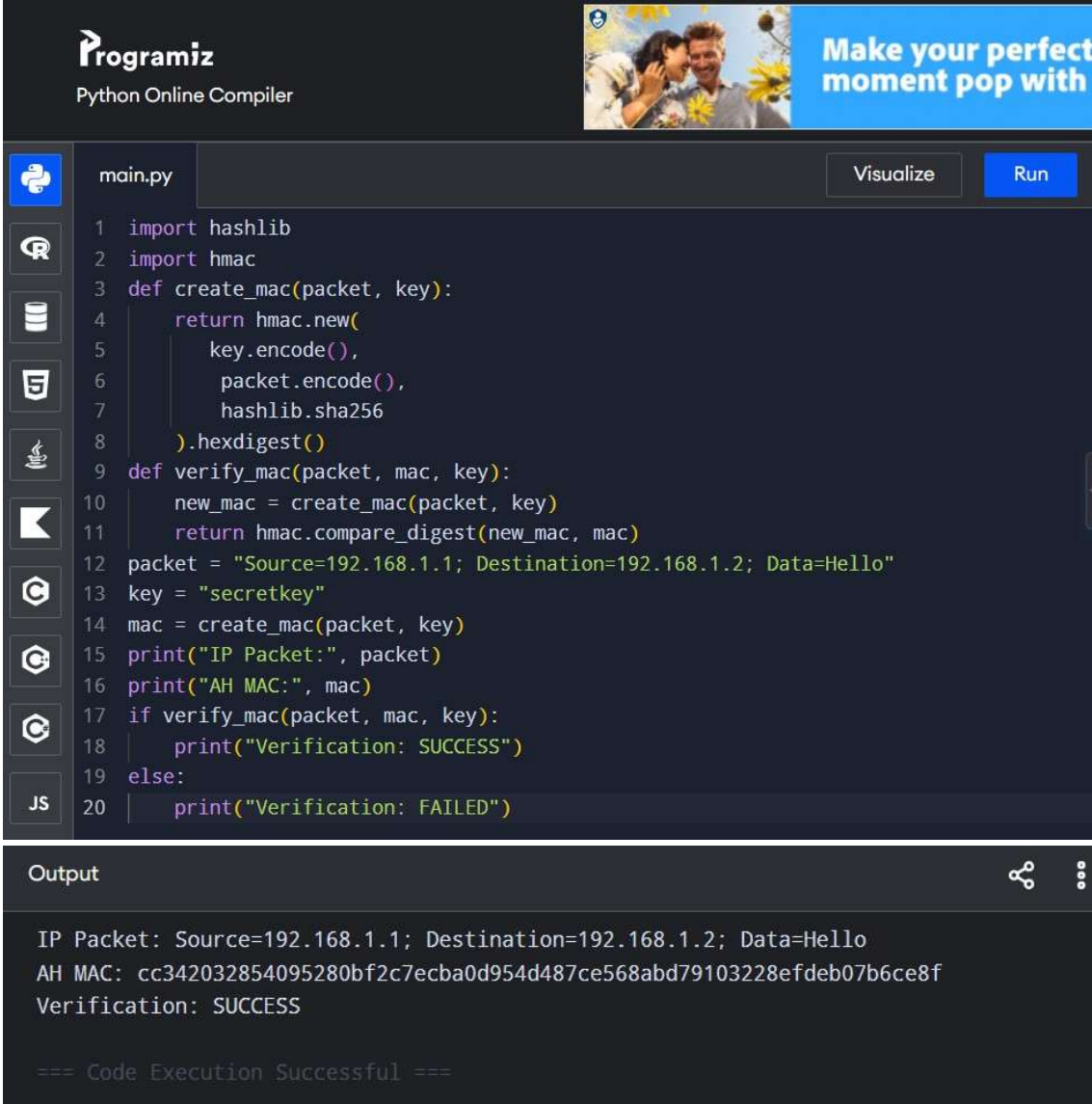
Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

ANSWER: IPSec Authentication Header (AH) – MAC Generation & Verification

Python Program:



The screenshot shows the Programiz Python Online Compiler interface. At the top, there's a header with the Programiz logo and a banner for a social media app. Below the header, the file name 'main.py' is displayed. The code editor contains a Python script that defines two functions: 'create_mac' and 'verify_mac'. The 'create_mac' function uses 'hashlib' and 'hmac' to generate a MAC for a given packet and key. The 'verify_mac' function compares the generated MAC with the provided MAC. The script then creates a packet string, a key, and a MAC, prints them, and finally verifies the MAC, resulting in a 'SUCCESS' message.

```
1 import hashlib
2 import hmac
3 def create_mac(packet, key):
4     return hmac.new(
5         key.encode(),
6         packet.encode(),
7         hashlib.sha256
8     ).hexdigest()
9 def verify_mac(packet, mac, key):
10     new_mac = create_mac(packet, key)
11     return hmac.compare_digest(new_mac, mac)
12 packet = "Source=192.168.1.1; Destination=192.168.1.2; Data=Hello"
13 key = "secretkey"
14 mac = create_mac(packet, key)
15 print("IP Packet:", packet)
16 print("AH MAC:", mac)
17 if verify_mac(packet, mac, key):
18     print("Verification: SUCCESS")
19 else:
20     print("Verification: FAILED")
```

The output section shows the following text:

```
IP Packet: Source=192.168.1.1; Destination=192.168.1.2; Data=Hello
AH MAC: cc342032854095280bf2c7ecba0d954d487ce568abd79103228efdeb07b6ce8f
Verification: SUCCESS
```

At the bottom, it says "=== Code Execution Successful ===".

Result: Thus, the IPsec AH process was successfully simulated using HMAC-SHA256 to generate and verify the MAC for IP packet integrity.

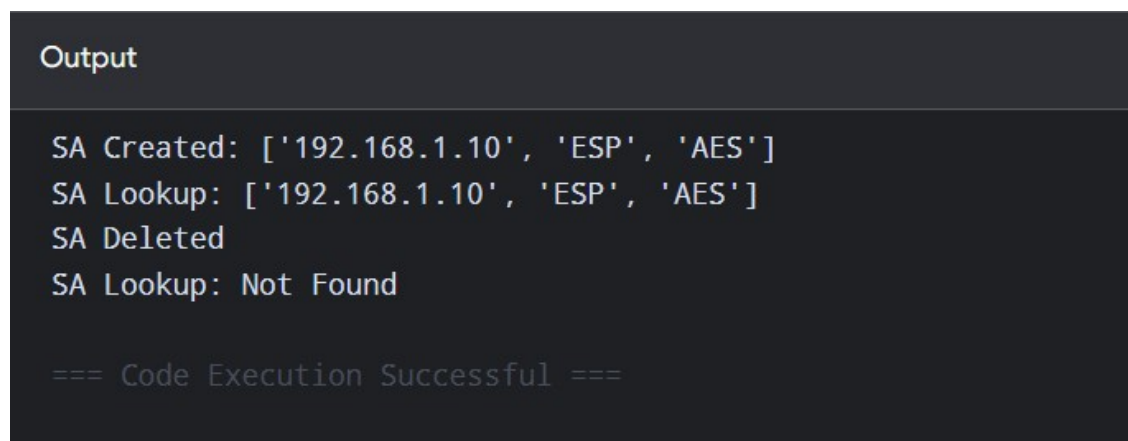
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

ANSWER: IPsec ESP – Encrypting & Decrypting IP Payload.

Python Program:



```
1 SA = {}
2 SA[101] = ["192.168.1.10", "ESP", "AES"]
3 print("SA Created:", SA[101])
4 spi = 101
5 print("SA Lookup:", SA.get(spi, "Not Found"))
6 del SA[101]
7 print("SA Deleted")
8 print("SA Lookup:", SA.get(101, "Not Found"))
```



```
Output

SA Created: ['192.168.1.10', 'ESP', 'AES']
SA Lookup: ['192.168.1.10', 'ESP', 'AES']
SA Deleted
SA Lookup: Not Found

=== Code Execution Successful ===
```

Result: ESP encryption and decryption was successfully implemented.

3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.

ANSWER: IPSec Security Association Management

Python Program:

```
main.py
1 def encrypt(data, key):
2     return ''.join(chr(ord(c) ^ key) for c in data)
3
4 payload = input("Enter payload: ")
5 key = 5
6
7 encrypted = encrypt(payload, key)
8 print("Encrypted:", encrypted)
9
10 decrypted = encrypt(encrypted, key)
11 print("Decrypted:", decrypted)
```

Enter payload: Hello
Encrypted: M`iij
Decrypted: Hello

...Program finished with exit code 0
Press ENTER to exit console.

Result: Security Association creation, lookup, and deletion were successfully implemented.

4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.

ANSWER: Simplified PGP System

Python Program:

```
main.py
1 message = "Secure Email"
2 p = 3
3 q = 11
4 n = p * q
5 e = 3
6 d = 7
7 data = [ord(c) for c in message]
8 encrypted = [(x ** e) % n for x in data]
9 decrypted = [(x ** d) % n for x in encrypted]
10 print("Original:", message)
11 print("Encrypted:", encrypted)
12 print("Decrypted:", ''.join(chr(x) for x in decrypted))
13 print("Digital Signature Created")

Original: Secure Email
Encrypted: [29, 8, 0, 24, 9, 8, 32, 27, 10, 25, 18, 3]
Decrypted:

Digital Signature Created

...Program finished with exit code 0
Press ENTER to exit console.
```

Result: A simplified PGP system was successfully implemented.

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

ANSWER: Simple Spam Email Classifier

Python Program:

```
main.py
1 spam = ["free", "win", "prize", "offer", "urgent"]
2
3 email = input("Enter email: ").lower()
4
5 count = 0
6
7 for word in spam:
8     if word in email:
9         count += 1
10
11 if count >= 2:
12     print("SPAM")
13 else:
14     print("LEGITIMATE")
```

Enter email: you won a free prize
SPAM

...Program finished with exit code 0
Press ENTER to exit console.

Result: The email was successfully classified as SPAM or LEGITIMATE using simple content-based filtering.